

The SDLC Experimentation Lab

One question drives the repo: *what software development life cycle can a six-person team plus current AI tools actually run?* This is the review of everything built so far — the apparatus, the method, two closed rounds, and the numbers.

10 tickets idea→deployed

24 PRs merged

2 rounds closed + readiness batch

\$21.72 Round 1, fully measured

0 false passes in-cycle

github.com/OlegKonyk/agentik-development

§ WHAT THIS IS

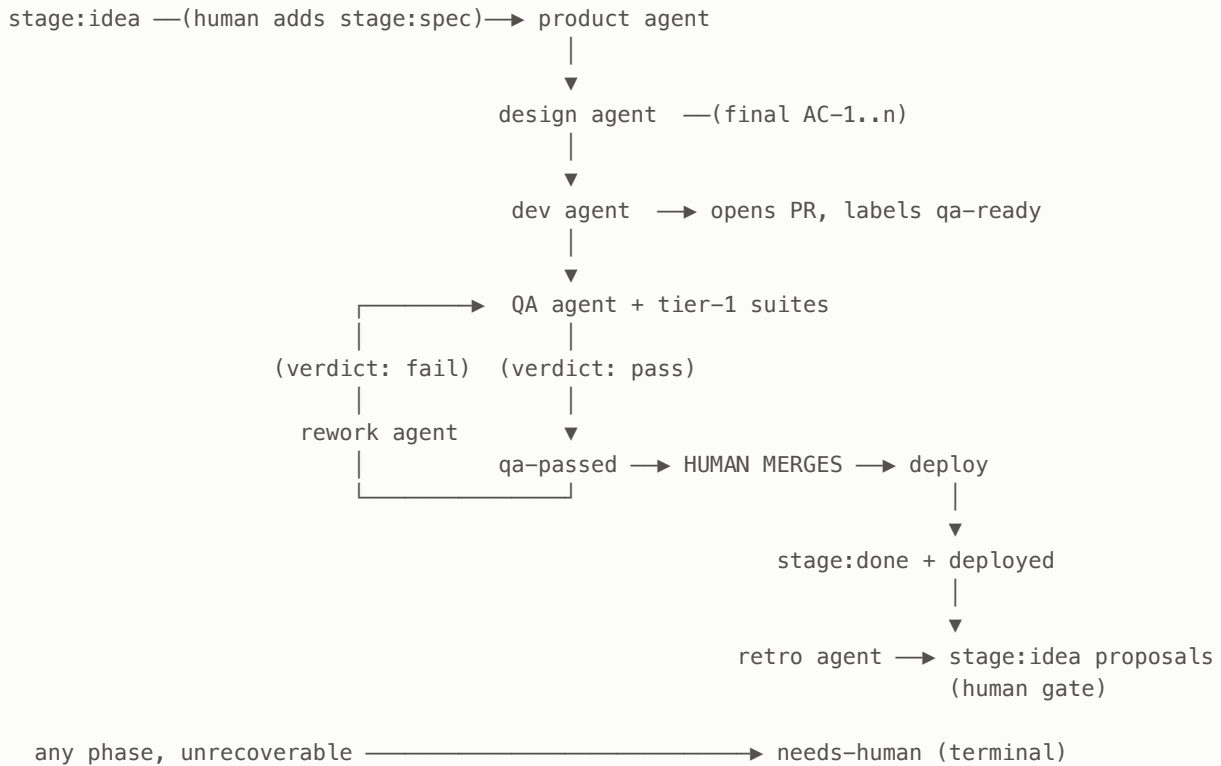
A pipeline that ships, measures itself, and critiques itself

The lab has three parts. The **apparatus** is a label-driven GitHub Actions state machine where Claude agents fill each phase — spec, design, implementation, QA verdict, rework, retrospective — and humans hold exactly the gates worth holding. The **specimen** is Taskboard: a deliberately boring task app on deliberately real architecture (Postgres, async jobs, signed webhooks, a programmable vendor, fault injection), so QA has genuine failure modes to hunt. The **method** is a charter that forces predefined ticket ledgers, forecast-vs-measured accounting, and a failure ledger with the same prominence as wins.

Nothing in the pipeline lets a model choose a state transition. Agents produce schema-validated content; deterministic YAML and scripts own every label, status, and merge gate. That split — *predictable orchestration, AI-filled content* — is the design thesis the whole experiment tests.

§ THE APPARATUS

The state machine



Evidence-gated merges

Branch protection requires `ci` and `qa/agent-verdict` on the exact head SHA. New code must earn a new verdict; a push after QA re-blocks the merge by construction.

The gate outranks the agent

Tier-1 suite red always refuses the pass, even when the QA agent argues the failure is out of scope. Verdicts without reproduction steps are downgraded, never trusted.

Three-layer privileged boundary

Agent charters, pipeline glue, and workflows are human-only — enforced by deny rules, a deterministic diff-and-dirty-tree guard before every agent push, and credential-less agent steps. Built after a rework agent walked through a dead deny rule and edited the QA charter.

Runaway and staleness guards

Per-phase turn and dollar caps, loop guards on every label cycle, and a freshness assert that refuses to spend a QA run on a stale merge ref (learned at a cost of \$1.80).

Full-spend telemetry

Every phase posts its cost line to the ticket (`Agent phase `qa` - 90 turns, ~$2.67`), so per-ticket economics are computed from GitHub events, not recollection.

The retro loop, gated

Every deployed ticket triggers a read-only retro agent: it drafts the learning-log entry and files ≤ 3 evidence-backed proposals. They enter the pipeline only when a human applies the entry label. The loop cannot close itself.

Six people, six changed jobs

Each seat has a playbook (docs/playbooks/) naming its touchpoints, its metrics, and what it must never delegate. Decision authority is single-owner by construction — the review found three seats each believing the same decision was exclusively theirs, and fixed it.

SEAT	OWNS	NEVER DELEGATES
Product manager	The spec as executable artifact; the entry gate for product tickets (stage:spec starts agent spend)	Answers to agent open questions; the final-AC review at design time
Dev manager	Merge authority over machine-verified state; cap and break-glass decisions; needs-human resolution	The merge, the bypass, reading QA evidence before shipping
Developers x2	Direction and review — agents draft every change; devs review with fresh eyes and route unfixable findings	Fresh-eyes review of agent diffs; reproducing findings before acting
Infra engineer	The paved road: workflows, guards, agent identities and least-privilege tokens, environments	Authoring the enforcement code that constrains agents; verifying it against its real runtime
QA engineer (lab owner)	The verification signal agents iterate against: tier-1 suites, the exploratory charter, gate rules, evidence formats	Adversarial review of agent-written tests; charter and gate changes

Four days, in order

JUL 24 · PROOF

The pipeline ships real work unattended

Ticket #1 flowed spec→design→dev→QA without a human touch. The reality-grade platform (#3) survived seven agent-QA cycles that found three real defects 131 deterministic tests and human review both missed. A bug ticket (#6) ran the loop fully autonomously in 17 minutes.

JUL 25 · HARDENING

Calibration tickets break the pipeline in instructive ways — and it gets fixed

The rework agent edited the QA agent's own charter through a dead permission rule; a QA run was spent testing a stale merge ref; a "surrogate 500" turned out to be three distinct

bugs. Result: PRs #15–#18 — the three-layer privileged boundary, the freshness assert, and the NUL/surrogate 422 fix, each adversarially reviewed before merge.

JUL 26 • FOUNDING

The repo becomes the lab — Round 0

Charter, five playbooks, the retro phase, metrics tooling, and a frozen baseline from the five pre-lab tickets (PR #19). The review that shipped it corrected the lab's own headline claim: the gate's escape rate is 1/5 at baseline, not zero. Per-phase cost telemetry landed the same day as the lab's first self-filed improvement (#20 → PR #21).

JUL 26 • ROUND 1

Three tickets through the full pipeline, every seat exercised

#4 board filter, #7 degraded-delivery banner (verified under live fault injection), #8 pagination — all merged through the honest gate, all deployed. One environment defect surfaced and was root-caused twice by the QA agent (gateway rate budget, fixed as #25). Four retro runs filed nine evidence-backed proposals; two were accepted and landed at round close.

JUL 27–28 • READINESS

Five accepted improvements land — and the pipeline turns up two defects in itself

The repeat-failure guard, capacity precheck, evidence rule, blocking-questions gate and retro dedupe shipped with 20 pinned tests now running in CI. En route: the dev phase was found to still carry the arg-scoped sandbox that PR #10 had fixed for QA alone, and dev-phase cost telemetry was discovered never to have worked — a silent 403 behind `|| true`. Both fixed. Ticket #36 then closed the contract tier's non-determinism by removing generation from the gating path entirely.

JUL 28 • BACKLOG CLEAR

The last open proposal ships, and the loop closes on itself

Ticket #31 gave QA the fault-injection lever it had asked for after being unable to prove an acceptance criterion black-box on #7 — **0.43 h, zero rework, zero parks, one human action** (the entry-gate label), and the first ticket in the lab's history measured with complete per-phase cost telemetry. Every retro proposal ever filed is now resolved, and the permission model that cost two phases a full run is an asserted invariant rather than a remembered lesson.

§ ROUND 1, MEASURED

Forecast difficulty inverted measured cost

The ledger and forecasts were committed before any ticket ran. The "trivial" control case cost 2.6× the "hardest" ticket — because it ran first into a latent environment defect. Feature

complexity predicted almost nothing; environment health predicted everything.

\$21.72

total agent spend, all
phases measured

1.88_h

pipeline wall clock
(forecast ≤7 h)

584

agent turns across 14
phase runs

2

unplanned parks —
both the same
environment defect

2/3

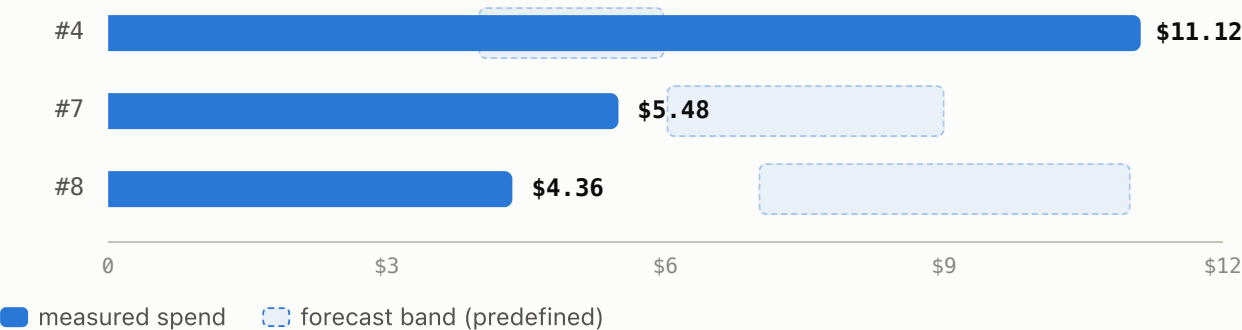
tickets passed QA
first-run

0

product defects found
in shipped features

Agent cost per ticket — measured against the predefined forecast band

Round 1, USD per ticket across all phases. Hover a bar for the ticket's story.



TICKET	VERDICT TRAIL	LEAD	TURNS	COST	REWORK	PARKS
#4 Filter the board the control case	× qa-failed × qa-failed ✓ qa-passed	0.85 h	326	\$11.12	2	2
#7 Degraded banner fault injection	✓ qa-passed	0.52 h	136	\$5.48	0	0
#8 Pagination multi-layer, rated hardest	✓ qa-passed	0.51 h	102	\$4.36	0	0
#20 Cost telemetry orchestrator lane	↳ bypass + CI	14 min	20	\$0.76	0	0

Round 0 baseline for comparison — five pre-lab tickets: 927 turns, \$27.64 with only QA phases counted (the undercount is documented and was itself the lab's first accepted improvement). Full definitions in `docs/lab-charter.md` ; regenerate any number with `uv run python scripts/lab_metrics.py` .

What the lab actually learned

Environment, not feature complexity, drove all cost variance

The e2e suite grew past the gateway's fixed rate budget and failed an unrelated test in two consecutive QA runs. Every layer behaved correctly — the QA agent root-caused it identically twice, rework correctly refused to "fix" a flake in app code, the gate refused honestly — **and the system still spent ~\$5.60 learning one fact**, because nothing could say "this failure class is not rework-fixable." Proposals #26/#28 exist to close that.

The gate has never false-passed in-cycle — and the honest escape rate is 1/5

Across every logged QA cycle the deterministic verdict layer has never emitted a false pass. One defect **did** escape a gate-passed merge (a NUL-byte 500 shipped inside ticket #3 and surfaced only when a later ticket's contract fuzzing happened to generate the input). The lab's own review caught the "zero" claim and corrected it — the metric reads 1/5 at baseline, stated everywhere.

The cause is now closed by construction: ticket #36 removed random generation from the gating path — the contract tier replays a version-controlled corpus, and generation became a human-run refresh producing a reviewable diff.

The design-wins rule survived a real disagreement

On #8 the PM called array-plus-header; design argued the envelope and out-reasoned the PM; the documented authority rule resolved it with no human override. The caveat is real too: the product→design transition never waits for PM answers, so alignment on the other two tickets was timing luck, not process guarantee.

Prompt-level boundaries are not boundaries

A rework agent once edited the QA agent's own charter because every `Edit(//ci/claude/**)` deny rule was dead — a path-syntax bug — and only one skill's prose forbade the path. The fix is layered and verified: working deny rules, a deterministic diff-and-dirty-tree guard at push time, credential-less agent steps. **Enforcement that exists only in prose is a suggestion.**

Verify enforcement code against the environment it runs in

The merge-ref freshness assert failed closed on its own first run — `rev-parse` cannot resolve parents in the depth-1 checkout CI actually uses. The rewrite was proven against a real shallow fetch before shipping. The lesson generalizes to every guard the lab writes.

A fix applied where a bug was found — not everywhere its shape exists — is a latent outage with a timer

PR #10 diagnosed that arg-scoped `Bash(...)` allowlists auto-deny the compound and env-prefixed commands agents naturally write, and that an agent burns its whole budget rephrasing instead of working. That was fixed **for QA only**. The dev phase carried the broken shape for the entire project, invisible because every prior dev ticket needed just `pytest`. Ticket #36 — the first needing a running API — burned 121 turns, \$6.25 and 13 denials on it. After the fix, the identical ticket and design completed in 101 turns, \$4.52, **zero denials**.

The retro loop reliably detects symptoms it cannot diagnose

The retro agent filed the same proposal three separate times (#27, #30, #33): dev-phase cost is missing from tickets. None of the three found the cause, because it was structurally invisible from the artifacts — `phase-dev.yml` granted `issues: read`, so every cost comment was a 403 swallowed by `|| true`. It had never worked, which is why Round 1's brief lists a dev cost as unmeasured. **That gap between noticing and diagnosing is the strongest practical argument for the human gate** the loop reports into.

The retro loop audits everyone, including its operator

Its first live run flagged that the lab's own first improvement shipped via the bypass route while the charter described only the full-pipeline route — a true charter-vs-practice gap, filed with evidence, accepted, and landed. Its known blind spot showed up the same day: three runs filed the same proposal (no cross-run memory), and the human gate absorbed the dedupe.

§ THE LOOP, IN PRODUCTION

Nine proposals from four retro runs

Every deployed ticket now produces its own evidence-backed critique. The queue below is the current state — acceptance is a human entry-gate decision, which is the whole point.

PROPOSAL	FROM	STATUS
#22 Charter names the bypass route for privileged-path work	retro #20	⬇ accepted · landed
#23 Metrics mark bypass-lane tickets distinctly	retro #20	⬇ accepted · landed
#26 Skip re-verification on repeat-identical infra flakes	retro #4	⌚ queued — recommended before Round 2
#28 Suite-size-aware capacity check before tier-1 e2e	retro #4	⌚ queued — recommended before Round 2
#27 Dev cost footer on the PR, matching other phases	retro #4	⌚ queued

#31 Finer-grained fault-injection lever for QA

retro #7

⌚ queued

#34 QA evidence must name coverage when substituting scenarios

retro #8

⌚ queued

#30, #33 Duplicates of #27 filed by later retro runs

retros #7, #8

⌚ closed as duplicates

§ OPEN DECISIONS

What waits on you

Round 2 is the team pilot. 21–30 day window, predefined ledger, all six people working their seats with playbooks in hand, forecast/perceived/measured per ticket. Everything Round 1 could not observe — handoff friction, review fatigue, authority conflicts between real people — is what Round 2 exists to measure.

Before it starts, three calls are yours:

- ① **Access grants.** Your five teammates cannot operate the pipeline without them: collaborator invites *accepted*, *write* per seat (the dev manager and both developers need merge), and the dev manager added as a *named bypass actor* rather than given admin.
- ② **The window dates**, and an acknowledgment that all six seats share your single subscription token — which is why the round runs tickets serially.
- ③ **The ledger contents** — nine candidate tickets with forecasts are drafted and fact-checked in `docs/case/round-2-ledger.md` ; it stays marked DRAFT until you confirm, because the charter forbids changing it once the window opens.

The proposal backlog is empty. Round 1's queue is fully resolved: #26, #28, #34 and the answer-window call all landed; #27 was closed by fixing its root cause, and the proposals the later retros filed (#37 orchestrator-lane cost, #42 permission-model audit, #31 fault-injection lever) have all shipped.

Standing limitation, stated as always: Round 1 had one person in all six seats and no control lane. The lab claims existence proofs and mechanisms — *this operating model ran, at this cost, with these failure modes* — not generalized productivity numbers. The perception-gap literature (METR: developers 19% slower while believing they were 20% faster) is why every claim above traces to pipeline telemetry, not to how the work felt.

Doc map — everything above is derived from these, in the repo:

`docs/lab-charter.md` — the method, binding

`docs/playbooks/` — five seat playbooks

`docs/case/round-0-baseline.md` — the frozen baseline

`docs/learning-log.md` — the raw record

`docs/sdlc.md` — the state machine spec

`docs/case/round-1-brief.md` — this round, one page

`docs/case/round-0-sources.md` — 46 cited sources

`docs/apps.md` — Taskboard contracts